FLASHDROP

Technical Implementation Blueprint

Android • Kotlin • Jetpack Compose • FDTP v1 • Hybrid P2P

Product direction	Fast, local, private peer-to-peer transfer
Platform	Android first release
Transfer architecture	FDTP application protocol over multiple transports
Design direction	Minimal dark interface

Prepared as the implementation baseline for product design, Android engineering, protocol development and release testing.

Document guide

- 1. Executive summary
- 2. Frozen product and technology decisions
- 3. Architecture principles
- 4. Project and module structure
- 5. FDTP v1 protocol specification
- 6. Connectivity and transport negotiation
- 7. Security design
- 8. Storage engine
- 9. Persistence and Room schema
- 10. Screen-by-screen UI plan
- 11. Background execution and Android permissions
- 12. Performance engineering
- 13. Testing and quality gates
- 14. Development roadmap and task breakdown
- 15. Release acceptance checklist
- 16. Engineering pitfalls to avoid
- 17. Recommended implementation order
- 18. Technical references

1. Executive summary

FlashDrop is a native Android peer-to-peer file transfer application designed for fast, private, offline transfers between nearby devices. It uses a transport-independent application protocol (FDTP) over multiple connection backends. The first release is Android-only, dark-first, and optimized for balanced throughput, reliability, compatibility and bounded resource use.

- No account or cloud relay is required for local transfers.
- The transfer engine is independent of the underlying wireless transport.
- Large files and multi-file batches are streamed with bounded memory.
- Transfers support approval, pause, interruption recovery, resume and integrity verification.
- Transport selection is capability-driven rather than hardcoded to one technology.

2. Frozen product and technology decisions

Product	FlashDrop
Package ID	com.flashdrop.android
Platform	Android first release
Language	Kotlin
UI	Jetpack Compose + Material 3
Architecture	Modular Clean Architecture + unidirectional data flow
State	Kotlin Flow / StateFlow
Dependency injection	Hilt
Persistence	Room + DataStore
Protocol	FDTP v1; Protocol Buffers for control messages
Primary transport	TCP over local network / Wi-Fi Direct
Additional transports	Wi-Fi Aware; Google Nearby Connections fallback
Security	TLS 1.3 on direct sockets; Nearby encrypted channel; authenticated pairing
Integrity	SHA-256
Storage access	Storage Access Framework
Background execution	Foreground service for active visible transfers; evaluate UIDT jobs where eligible
Design	Minimal dark interface
Cloud/account	Not required

Set the minimum SDK after validating the selected Wi-Fi Direct, Wi-Fi Aware, Nearby and background-execution APIs. API 26 can be an initial engineering baseline, but release compatibility must be checked against current Android and Google Play requirements.

3. Architecture principles

- Separate the FDTP protocol from transport adapters and UI features.
- Keep feature modules dependent on domain interfaces, not socket or Wi-Fi implementation details.
- Use bounded streaming and backpressure; never load an entire file into memory.
- Persist meaningful state transitions and verified checkpoints, not every packet.
- Treat each transport as a connection provider that carries the same logical FDTP session.
- For v1, reconnect and resume when changing transport; do not attempt transparent mid-stream transport migration.

System layers: Compose UI → ViewModels → domain use cases → repositories/session manager → FDTP engine → transport adapter → local wireless link.

4. Project and module structure

```
FlashDrop/
app/
core/
common/ model/ database/ datastore/
security/ permissions/ designsystem/
domain/
repository/ usecase/
data/
repository/ mapper/
feature/
home/ send/ receive/ discovery/ pairing/
transfer/ history/ filebrowser/ settings/
connectivity/
api/ discovery/ lan/ wifip2p/ wifiaware/ nearby/
protocol/
api/ serialization/ session/ control/
streaming/ scheduler/ resume/ verification/
storage/
api/ source/ destination/ manifest/ filesystem/
service/
transfer/
benchmark/
protocol/ throughput/ device/
```

Module	Responsibility
app	Application startup, navigation and dependency graph
core:model / common	Shared models, result types, utilities
core:database	Room entities, DAOs and migrations
core:security	Keystore identity, cryptographic helpers and pairing
core:permissions	Version-aware permission checks and explanations
core:designsystem	Dark theme, typography, colors and reusable components
feature:*	Independent Compose screens and ViewModels
domain	Business rules, use cases and repository contracts
data	Repository implementations and mappings
connectivity:api	Transport and connection interfaces
connectivity:lan	NSD discovery and TCP/TLS connections
connectivity:wifip2p	Wi-Fi Direct peer discovery and group setup
connectivity:wifiaware	Wi-Fi Aware publish/subscribe and network attachment
connectivity:nearby	Nearby Connections advertising, discovery and payload bridge
protocol	FDTP framing, handshake, manifest, streaming, ACK, resume and verification
storage	SAF source/destination adapters and safe file operations
service:transfer	Foreground service, notifications and recovery lifecycle
benchmark	Throughput, memory, thermal and device test harness

Dependency rule: feature modules must not depend directly on connectivity implementations or low-level FDTP classes. They invoke domain use cases and observe state exposed by repositories.

5. FDTP v1 protocol specification

FDTP (FlashDrop Transfer Protocol) is the application-level protocol shared by all transports. It defines session negotiation, peer authentication, manifests, file/chunk identity, acknowledgements, recovery, cancellation and completion.

5.1 Goals and versioning

- Binary control messages; no dependence on JSON property ordering.
- Persistent sessions and multi-file manifests.
- Large-file support with bounded memory and bounded message sizes.
- Resume based only on receiver-verified and committed data.

- Major-version mismatch: reject; minor-version mismatch: negotiate compatible capabilities.
- Unknown optional fields may be ignored; unsupported mandatory features must fail explicitly.

5.2 Protocol stack

Application → FDTP session/control/data/resume services → transport adapter → local wireless link. Direct socket transports use TCP + TLS 1.3. Nearby Connections does not expose a raw TCP socket: the adapter must carry FDTP logical messages through Nearby byte, stream or file payloads. Do not attempt to create a second TCP/TLS layer over Nearby's encrypted connection.

5.3 Session handshake

Message	Purpose
CLIENT_HELLO	Protocol version, client nonce, ephemeral identity, capabilities and limits
SERVER_HELLO	Selected version, server nonce, capabilities and challenge
SESSION_AUTH	Authentication proof and pairing confirmation
AUTH_RESULT	Authentication outcome
SESSION_INIT	Requested session parameters
SESSION_READY	Session ID, negotiated limits and resume support
TRANSFER_MANIFEST	File and directory metadata
TRANSFER_ACCEPT / REJECT	Receiver decision
TRANSFER_START	Begin data transfer
DATA_CHUNK	File content at a declared offset
CHUNK_ACK	Receiver-confirmed committed range
FILE_END / FILE_VERIFIED	File boundary and integrity result
TRANSFER_PAUSE / RESUME	Pause or resume session
RESUME_STATE	Receiver's verified chunk state
TRANSFER_CANCEL	Cancel transfer
TRANSFER_COMPLETE	Session completion
ERROR; PING / PONG	Errors and connection health

5.4 TCP frame format

Field	Size
Magic (`FDRP`)	4 bytes
Version	1 byte
Type	1 byte
Flags	2 bytes
Payload length	4 bytes
Sequence	4 bytes
Total header	16 bytes

All multi-byte integers use big-endian byte order. Validate the declared length before allocating or reading the payload. Set a strict maximum control payload (initially 1 MiB). Data payload limits are separately negotiated. Unknown frame types must be rejected unless explicitly designated as ignorable extensions. This TCP frame envelope is not applied directly to Nearby payloads; the Nearby adapter maps the same logical messages to its payload API.

5.5 Manifest model

```
syntax = "proto3";
package flashdrop.protocol.v1;

message TransferManifest {
string session_id = 1;
repeated FileEntry files = 2;
uint64 total_bytes = 3;
uint32 total_files = 4;
}

message FileEntry {
string file_id = 1;
string file_name = 2;
string relative_path = 3;
uint64 file_size = 4;
string mime_type = 5;
bytes sha256 = 6;
uint32 chunk_size = 7;
uint64 total_chunks = 8;
FileType type = 9;
}

enum FileType {
FILE_TYPE_UNSPECIFIED = 0;
REGULAR_FILE = 1;
DIRECTORY = 2;
}
```

Production extensions: manifest pagination, unique file IDs, explicit directory entries, path and duplicate validation, strict manifest limits, and capability negotiation. MIME type is advisory and must never be treated as a security guarantee.

5.6 Chunking and flow control

[illegible]	[illegible]
Default chunk	4 MiB
Minimum chunk	256 KiB
Benchmark candidates	8 MiB; 16 MiB
Initial data streams	1
Concurrent file operations	2
Maximum concurrent files	4
Memory budget	32–64 MiB, bounded

Each chunk is associated with session ID, file ID, chunk index, file offset and payload length. The receiver validates session/file identity, expected offset, exact length, overlap, declared file size and sequence. Use a bounded sliding window and cumulative acknowledgements for receiver-committed data. Avoid reimplementing TCP retransmission; FDTP ACKs exist for application-level commit, progress and recovery.

5.7 Resume

- Receiver records verified chunk ranges and commits checkpoints periodically.
- After reconnection, peers re-authenticate and validate session and manifest identity.
- Receiver reports only data that is verified and safely committed.
- Sender retransmits missing ranges; already committed ranges are not resent.
- Use bounded retry/backoff and expire stale sessions according to a retention policy.

5.8 State machine

Idle → Discovering → Connecting → Authenticating → Negotiating → AwaitingApproval → Transferring → Verifying → Completed. Alternate states include Rejected, Paused, Interrupted, Reconnecting, Failed, Retrying and Cancelled. Persist state transitions so the UI and service can recover consistently after process death.

6. Connectivity and transport negotiation

6.1 Transport abstraction

```
interface TransferTransport {
val transportType: TransportType
suspend fun isAvailable(): Boolean
suspend fun discover(): Flow<RemoteDevice>
suspend fun connect(device: RemoteDevice): TransportConnection
suspend fun disconnect()
fun capabilities(): TransportCapabilities
}

interface TransportConnection {
val connectionState: StateFlow<ConnectionState>
suspend fun sendControl(message: ControlMessage)
suspend fun openDataWriter(): DataWriter
fun receiveMessages(): Flow<TransportMessage>
suspend fun close()
}
```

This is a conceptual contract. Production APIs should stream data through bounded readers/writers, not pass large files as ByteArray values. Define cancellation, backpressure, ownership, close semantics and error propagation explicitly.

6.2 Adapters

Transport	Role and implementation
LAN TCP	Primary when devices share a local network; NSD discovery, TCP socket and TLS 1.3
Wi-Fi Direct	Primary no-router path; peer discovery, group negotiation, local endpoint and TLS
Wi-Fi Aware	Optional on supported hardware; publish/subscribe, peer matching and network attachment
Nearby Connections	Compatibility fallback; encrypted connection and FDTP logical messages carried as Nearby payloads

6.3 Selection policy

- Prefer a successfully established LAN path when both devices share a usable local network.
- Use Wi-Fi Direct when no shared router is available and both devices support it.
- Consider Wi-Fi Aware where both devices support it and measured performance is suitable.
- Use Nearby Connections when direct setup is unavailable or unreliable.
- Choose using capabilities, connection success, measured throughput, stability and setup cost—not theoretical speed alone.
- For v1, transport changes require reconnect + FDTP resume.

6.4 Discovery

Advertise only service ID, protocol major version, ephemeral device ID, display name, supported transport mask and connection nonce. Do not broadcast phone numbers, serial numbers, permanent identifiers or file information. Deduplicate results after authenticated identity is established; never trust an unauthenticated display name.

7. Security design

Concern	Recommendation
Socket confidentiality	TLS 1.3
Nearby confidentiality	Nearby encrypted connection
Device identity	Keystore-backed asymmetric key pair
Pairing	Authenticated short-code comparison and explicit user confirmation
File integrity	SHA-256 per completed file; optional chunk-level checks
Random values	Cryptographically secure random generator
Trusted devices	Public-key fingerprints; private keys remain in Android Keystore

7.1 Receive-side safeguards

- Require explicit user approval for incoming transfers unless a narrowly scoped trusted-device policy is enabled.

- Write to temporary destinations and commit only after validation.
- Reject absolute paths, traversal segments, invalid names and duplicate destination paths.
- Enforce manifest, file-count, per-file and total-size quotas.
- Check free space before acceptance and handle provider-specific storage failures.
- Never automatically open, execute or install received files.
- Do not trust file extensions or MIME type as proof of content.

7.2 Threat model

Address unknown senders, spoofed identities, interception, tampered content, malformed frames, oversized manifests, path traversal, replay, memory exhaustion and storage exhaustion. Use authenticated sessions, nonces, strict parser limits, quotas, bounded buffers and verified checkpoints. Do not invent cryptographic algorithms.

8. Storage engine

Use Android's Storage Access Framework and ContentResolver. Do not assume all selected items have ordinary filesystem paths; cloud-backed and virtual document providers may expose content URIs. Prefer file descriptors/channels where supported and use streaming I/O.

```
interface FileSource {
suspend fun metadata(uri: Uri): FileMetadata
suspend fun openRead(uri: Uri): SourceReader
}

interface FileDestination {
suspend fun createTemporaryFile(
relativePath: String, expectedSize: Long
): DestinationHandle
suspend fun openWriter(handle: DestinationHandle): DestinationWriter
suspend fun finalizeFile(handle: DestinationHandle): FinalizedFile
suspend fun discard(handle: DestinationHandle)
}
```

Suggested default destination: Download/FlashDrop/. Preserve the sender's relative folder structure inside the chosen destination. Use temporary files, validate size and SHA-256, then atomically rename where the destination provider supports it. Handle SAF providers that do not support atomic rename explicitly.

9. Persistence and Room schema

Entity	Key fields
TransferSession	sessionId, remoteDeviceId/name, direction, transport, status, totalBytes, transferredBytes, createdAt, updatedAt
TransferFile	fileId, sessionId, fileName, relativePath, fileSize, mimeType, sha256, status, completedBytes, temporaryUri, destinationUri
TransferCheckpoint	fileId, checkpointVersion, chunkSize, verifiedRanges, committedBytes, updatedAt
TrustedDevice	deviceId, displayName, publicKeyFingerprint, trustedAt, lastSeenAt

- Persist meaningful state transitions and periodic checkpoints.
- Batch database writes; do not write every packet or UI progress tick.
- Never store file contents or private keys in Room.
- Keep history separate from temporary transfer data.
- Use migrations and define cleanup/retention rules for stale partial sessions.

10. Screen-by-screen UI plan

Visual direction: dark-first Material 3, charcoal surfaces, restrained violet accent, clear typography, compact cards, subtle borders and minimal motion. Suggested tokens: background #101216; surface #191D24; elevated #232936; accent #A78BFA; primary text #F1F3F8; secondary text #969EAD; success #4ADE80; error #F87171.

Screen	Main components and actions
Home	Brand header, Send, Receive, recent transfers, visibility indicator

Screen	Core components and actions
File Picker	Categories, folders, search, sort, multi-select, selected-size summary
Device Discovery	Scanning state, nearby devices, availability, QR/manual pairing
Pairing	Peer names, matching verification code, trust option, confirm/cancel
Incoming Transfer	Sender identity, file/folder summary, total size, destination, accept/reject
Active Transfer	Overall and per-file progress, speed, ETA, pause/cancel, current file
Completion	Success/partial failure summary, total bytes, average speed, open destination
History	Date groups, direction, peer, size, status, details/retry
Settings	Device name, visibility, save location, trusted devices, notifications, diagnostics

10.1 Navigation

Home → Send → File Picker → Device Discovery → Pairing → Active Transfer → Completion. Home → Receive → Incoming Request → Pairing/Approval → Active Transfer. Home also links to History and Settings.

10.2 UI state

```
data class TransferUiState(
val status: TransferStatus,
val totalBytes: Long,
val transferredBytes: Long,
val speedBytesPerSecond: Long,
val remainingSeconds: Long?,
val files: List<FileTransferUiModel>,
val canPause: Boolean,
val canCancel: Boolean
)
```

Use unidirectional flow: UI action → ViewModel → use case → repository/transfer manager → engine → StateFlow → ViewModel → Compose. Publish throttled UI progress (about 4–5 updates/second) while the engine continues independently. Show ETA only after enough stable throughput samples exist.

11. Background execution and Android permissions

Use a foreground service for an active, user-visible transfer where the current Android version and service rules permit it. Evaluate User-Initiated Data Transfer (UIDT) jobs for eligible long-running user-requested work. Handle service timeouts, process death, notifications, checkpointing and restart/recovery explicitly. Android version and target-SDK rules change; verify current requirements before release.

Capability	Permission or API area
Network sockets	INTERNET
Wi-Fi state/configuration	ACCESS_WIFI_STATE, CHANGE_WIFI_STATE as applicable
Wi-Fi Direct / Aware	NEARBY_WIFI_DEVICES and version-specific requirements
Bluetooth discovery	BLUETOOTH_SCAN / CONNECT / ADVERTISE if used
Foreground transfer	FOREGROUND_SERVICE and required service-type permission
Notifications	POST_NOTIFICATIONS where applicable
Files	Storage Access Framework; avoid broad storage access

Do not request every permission at first launch. Explain why a permission is needed at the point of use. Add legacy location permissions only where required by older platform APIs. Include future local-network permission changes in the Android target-SDK migration checklist. The manifest is release-specific and must be validated against the selected target SDK and Play policy.

12. Performance engineering

Parameter	Starting baseline
Active sessions	1
Concurrent file operations	2
Data streams per socket	1
Chunk size	4 MiB default; benchmark 256 KiB–16 MiB
Memory budget	32–64 MiB bounded
UI progress	4–5 Hz
Checkpoint interval	1–5 seconds, tuned to durability and overhead
Reconnect	Bounded exponential backoff

Measure network throughput, receiver write throughput, buffer occupancy, CPU, thermal state and connection stability. Apply backpressure when storage is slower than the network. Reduce in-flight work on weak links or thermal throttling. Increase bounded windows only when benchmarks show improvement. Compression should be optional and content-aware; most video, image and archive files are already compressed.

12.1 Release engineering targets

Area	Provisional acceptance target
LAN throughput	At least 70% of measured usable baseline on reference test setup
Wi-Fi Direct throughput	At least 70% of measured usable baseline on reference test setup
Memory	Bounded regardless of file size
Large-file design target	100 GB+ where source, destination and device permit
Resume	No retransmission of receiver-verified committed chunks
Integrity	All successfully completed files verified
Discovery	Median target under 3 seconds in defined test conditions
Connection	Median target under 5 seconds in defined test conditions

These are engineering targets, not guaranteed user-facing performance claims. Measure at least five runs per scenario and report median and 95th percentile. Record device model, OS build, Wi-Fi band, distance, signal conditions and storage type.

13. Testing and quality gates

13.1 Unit tests

- Protocol: frame encoding/decoding, malformed packets, version negotiation, sequence and manifest validation.
- Scheduler: multi-file queue, bounded concurrency, backpressure, pause/resume, cancellation and retry.
- Storage: large writes, Unicode names, duplicates, insufficient space, interrupted writes and finalization.
- Security: invalid pairing, authentication failure, tampered content, path traversal and oversized frames.

13.2 Integration tests

Two devices on same router	LAN discovery and transfer
No router	Wi-Fi Direct transfer where supported
Compatible Wi-Fi Aware devices	Successful Aware session
Nearby fallback	FDTP transfer over Nearby payload channel
Screen locked/backgrounded	Transfer behaves within OS execution rules
Network disconnected	Session enters interrupted state
Network restored	Resume from verified checkpoints
Storage full	Graceful failure with recoverable state
Process killed	Recoverable session/partial file
Duplicate filename	No silent overwrite
Modified data	Integrity failure and no final commit

13.3 Stress and device matrix

- Test one 20 GB file and, where hardware permits, a 100 GB file.
- Test 1,000 mixed files, 10,000 small files and deeply nested folders.
- Interrupt at 25%, 50% and 90%; repeat pause/resume cycles.
- Test under CPU load, low storage and thermal stress.
- Cover Xiaomi/Redmi/POCO, Samsung, OnePlus, Pixel and Motorola across low-, mid- and flagship tiers.

A test passes only when destination contents and checksums are correct, persisted state is consistent, memory remains bounded and no unsafe overwrite or path escape occurs.

14. Development roadmap and task breakdown

0 — Foundation	3–4 days	Project, Gradle catalog, modules, Hilt, Room, Compose theme, Git/CI. Deliverable: clean build.
1 — UI & files	5–7 days	Navigation, home, picker, SAF, folder selection, multi-select, metadata, settings. Deliverable: usable UI shell.
2 — LAN	5–7 days	NSD, TCP server/client, lifecycle, health checks. Deliverable: two devices connect locally.
3 — FDTP	7–10 days	Protobuf, framing, handshake, manifest, chunks, ACK, verification. Deliverable: first complete transfer.
4 — Reliability	7–10 days	Bounded streaming, scheduler, multi-file, progress, pause/resume, checkpoints, recovery. Deliverable: resilient transfer engine.
5 — Hybrid transports	7–10 days	Wi-Fi Direct, Wi-Fi Aware, Nearby adapters, capability detection, fallback. Deliverable: multiple connection methods.
6 — Security & background	5–7 days	TLS, identity, pairing, approval, foreground service, notifications, secure temp files. Deliverable: secure background operation.
7 — Optimization & release	7–10 days	Benchmarks, profiling, thermal/device testing, accessibility, signing and release candidate.

Indicative total: approximately 7–11 weeks for an experienced Android developer working primarily on the project. Device-specific integration, OEM restrictions, test hardware and release compliance can extend the schedule.

15. Release acceptance checklist

- Two Android devices discover each other and transfer in both directions.
- Multi-file and folder transfers preserve names and hierarchy.

- Large files transfer with bounded memory.
- Interrupted transfers resume without retransmitting verified committed data.
- Pause, cancel, duplicate handling and storage failure work safely.
- All completed files pass integrity verification.
- Socket paths are encrypted and pairing is authenticated.
- Incoming transfers require approval under default settings.
- Background operation follows current Android restrictions and shows a clear notification.
- Dark UI, accessibility, permission rationale and history are complete.
- No account, cloud relay or internet connection is required for local transfer.

16. Engineering pitfalls to avoid

Loading full files into RAM	Bounded streaming and backpressure
Reconnecting for every file	Persistent session/connection
Unlimited concurrent chunks	Bounded scheduler
Hardcoded transport choice	Capability-driven negotiation
UI driven by socket callbacks	StateFlow and unidirectional data flow
Trusting sender filenames	Strict path validation
Re-sending all data after interruption	Receiver-verified checkpoints
Trusting device display name	Authenticated cryptographic identity
Assuming all devices support Wi-Fi Aware	Runtime capability detection
Transparent transport migration in v1	Reconnect and resume

17. Recommended implementation order

1. Build the Android foundation: Kotlin, Compose, navigation, permissions and storage.
2. Build LAN connectivity: discover two devices and establish a secure connection.
3. Implement FDTP: handshake, manifest, chunk streaming, acknowledgements and verification.
4. Prove the engine with 2 GB, 10 GB and multi-file test transfers.
5. Add recovery and security: pairing, encryption, checkpoints and background execution.
6. Add Wi-Fi Direct, Wi-Fi Aware and Nearby transport adapters.
7. Polish the dark UI, optimize performance and complete manufacturer testing.

First milestone: two Android phones from different manufacturers discover each other on the same Wi-Fi network, establish a secure FDTP session, transfer a 10 GB file, display accurate throughput and verify the destination checksum.

18. Technical references

Use the current official Android and Google documentation during implementation. Verify API availability, permission behavior, foreground-service restrictions and Play policy at the time of release.

- Android Wi-Fi Direct: https://developer.android.com/develop/connectivity/wifi/wifip2p
- Android Wi-Fi Aware: https://developer.android.com/develop/connectivity/wifi/wifi-aware
- Google Nearby Connections: https://developers.google.com/nearby/connections/overview
- Android foreground services: https://developer.android.com/develop/background-work/services/foreground-services
- User-initiated data transfer jobs: https://developer.android.com/develop/background-work/background-tasks/uidt
- Storage Access Framework: https://developer.android.com/guide/topics/providers/document-provider